

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 3 SEPTEMBER 2019 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

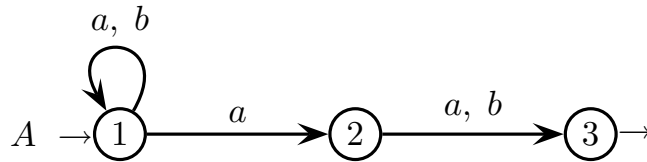
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A below, over the two-letter alphabet $\Sigma = \{a, b\}$:



Answer the following questions:

- (a) List all the strings of length 3 that are accepted by automaton A , and those (still of length 3) that are rejected by A .
 - (b) Use the Berry-Sethi (BS) method to derive a deterministic automaton A_d equivalent to automaton A . By using a systematic method, check whether automaton A_d is minimal, and if necessary minimize it.
 - (c) By using a systematic method, obtain an automaton $\overline{A}$ that accepts language $\overline{L(A)}$, i.e., the complement of language $L(A)$. List the accepting computations of $\overline{A}$ for the strings of length 3, which are rejected by automaton A .
 - (d) (optional) By using the node elimination (BMC) method, find a regular expression R that generates language $L(A)$. Based on R , provide an informal, i.e., written in English, concise and precise description of language $L(A)$.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the ambiguous grammar G below, over a two-letter terminal alphabet $\{ a, b \}$ (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S b S \\ S \rightarrow a S \\ S \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Verify that grammar G is ambiguous, by finding a short string that has (at least) two syntax trees, and draw these two trees.
- (b) Consider the following grammar G' , similar to G but not identical (axiom S):

$$G' \left\{ \begin{array}{l} S \rightarrow S a S b \\ S \rightarrow S b \\ S \rightarrow \varepsilon \end{array} \right.$$

Say if grammar G' is ambiguous and reasonably justify your answer.

- (c) (optional) Is the intersection language $L(G) \cap L(G')$ regular? Is it context-free? Justify your answers.
-

2. A programming language fragment includes the expressions and variables that represent sets of binary integers, and the assignments of such expressions to the variables, according to these rules:
 - A set expression can be one of the following:
 - a variable name, i.e., an identifier, modeled by the terminal symbol *id*, not to be further expanded
 - a list, possibly empty, of comma-separated binary integer constants; the list is enclosed within a pair of curly brackets ‘{ }’; a binary integer constant may not have any leading zeros
 - the union (keyword *union*) or the intersection (keyword *inters*) or the complement (keyword *compl*) of set (sub)expressions; union and intersection are binary operators and are *right*-associative, while complement is unary; by default the precedence between these set operators is, in decreasing order: complement, intersection and union
 - a pair of round brackets ‘()’ that enclose a set (sub)expression; as usual, by using round brackets one can change the default precedence defined between the set operators, see the fourth assignment in the sample sentence below
 - An assignment statement, with the operator ‘:=’, assigns a set expression (on the right-hand side) to a variable (on the left-hand side).
 - A sentence of the language is a non-empty list of semicolon-separated assignment statements.

Here is a sample sentence of the language (with four assignments):

```
s1 := { 101, 1001 };

s2 := { 11 } union s1;

s3 := compl s2 inters s1 union { 110 } union { 10 };

s2 := compl ( s1 union compl { 100, 10 } )
```

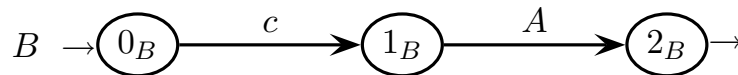
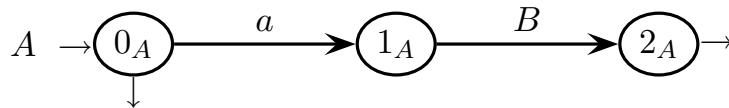
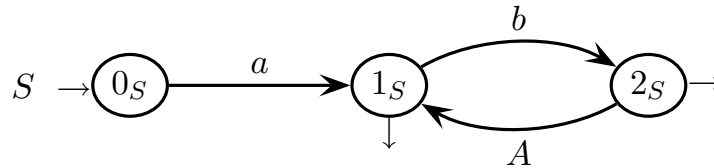
Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the above sketched language.
- (b) Test the correctness of your grammar by drawing the syntax subtree of the expression in the third assignment statement in the sample sentence above, i.e., the subtree of the expression:

```
compl s2 inters s1 union { 110 } union { 10 }
```


3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, over a three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and a three-letter nonterminal alphabet $V = \{ A, B, S \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) By drawing the complete pilot, determine whether grammar G is of type $ELR(1)$. Justify your answer.
- (b) By finding all the guide sets on the call arcs and exit arrows, determine whether grammar G is of type $ELL(1)$. Justify your answer.
- (c) By means of the Earley algorithm, analyze the valid string:

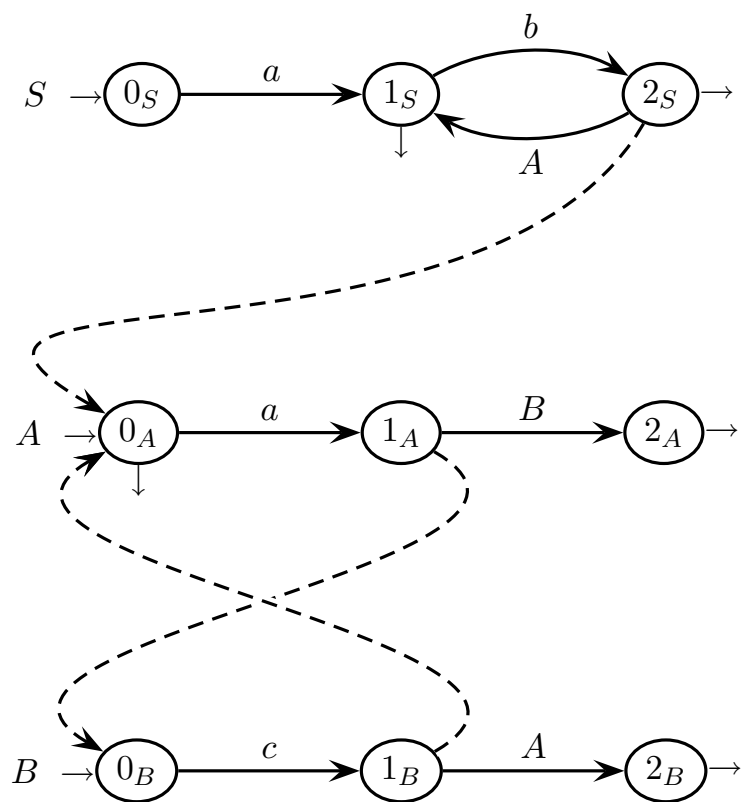
$$a b a c \in L(G)$$

and draw its syntax tree.

- (d) (optional) Is language $L(G)$ regular ? Justify your answer.

please draw here the pilot – question (a)

please write here the guide sets – question (b)



space for answering question (c)

Earley vector of string $a b a c$ (to be filled in – the number of rows is not significant)

[illegible]

please draw here the tree of question (c) and answer question (d)

free space for any use

4 Language Translation and Semantic Analysis 20%

1. The source grammar G_s below (axiom S), over the two-symbol terminal alphabet $\Sigma = \{ '(', ')' \}$, generates as source language L_s the Dyck language over round parentheses $'('$ $)'$:

$$G_s \left\{ \begin{array}{l} S \rightarrow (S) T \\ S \rightarrow \varepsilon \\ T \rightarrow (S) S \\ T \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Write a syntax scheme G_τ (or a translation grammar) for the translation τ , over the source language L_s , defined as follows. In all the sequences of consecutive matching pairs of parentheses at any nesting level (including the top one):
 - the matching pairs that occur at odd positions, i.e., the first, third, fifth and so on, are left unchanged
 - the matching pairs that occur at even positions, i.e., the second, fourth, sixth and so on, are changed into square brackets

Two examples are reported below. For clarity, the bracket pairs of the first example are outlined, distinguishing the pairs that are left unchanged (underlined) from those that are changed into square brackets (overlined):

$$\tau \left(\underbrace{ \left(\underbrace{ () }_{\text{unchanged}} \overbrace{ (()) }^{\text{changed}} \right) }_{\text{unchanged}} \overbrace{ () }^{\text{changed}} \right) = \left(\underbrace{ () }_{\text{unchanged}} \overbrace{ [()]] }^{\text{changed}} \right) \overbrace{ [] }^{\text{changed}}$$

and

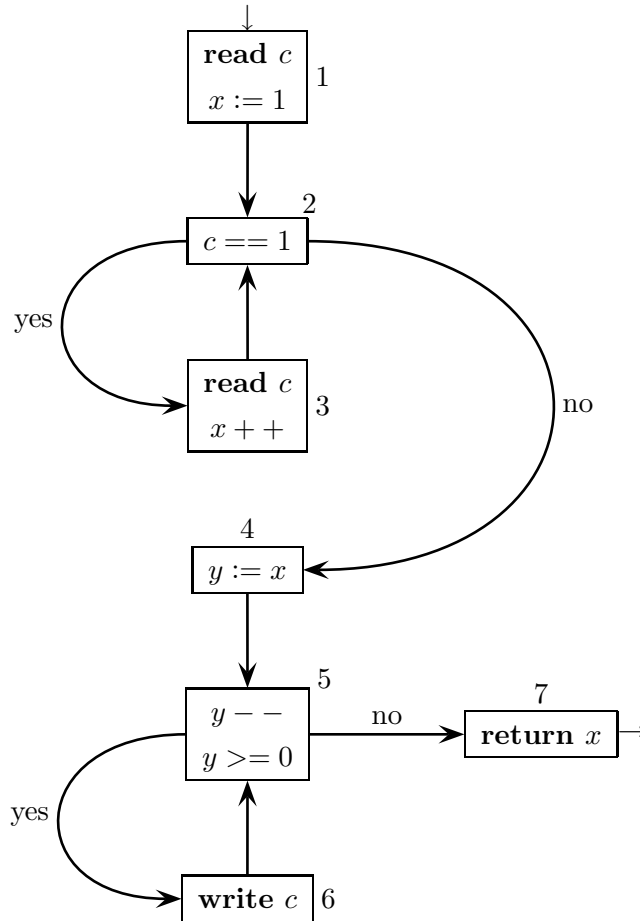
$$\tau \left((((() ()) (() ()) ()) ((() ())) \right) = (((() [] ()) [() []] ()) [(() []])$$

Do not change the source grammar G_s .

- (b) Draw the syntax translation tree of the valid source string of the first example above, i.e., the source string:

$$((() (() ())) ()$$

2. Consider the program Control Flow Graph (*CFG*) shown below, with seven nodes:



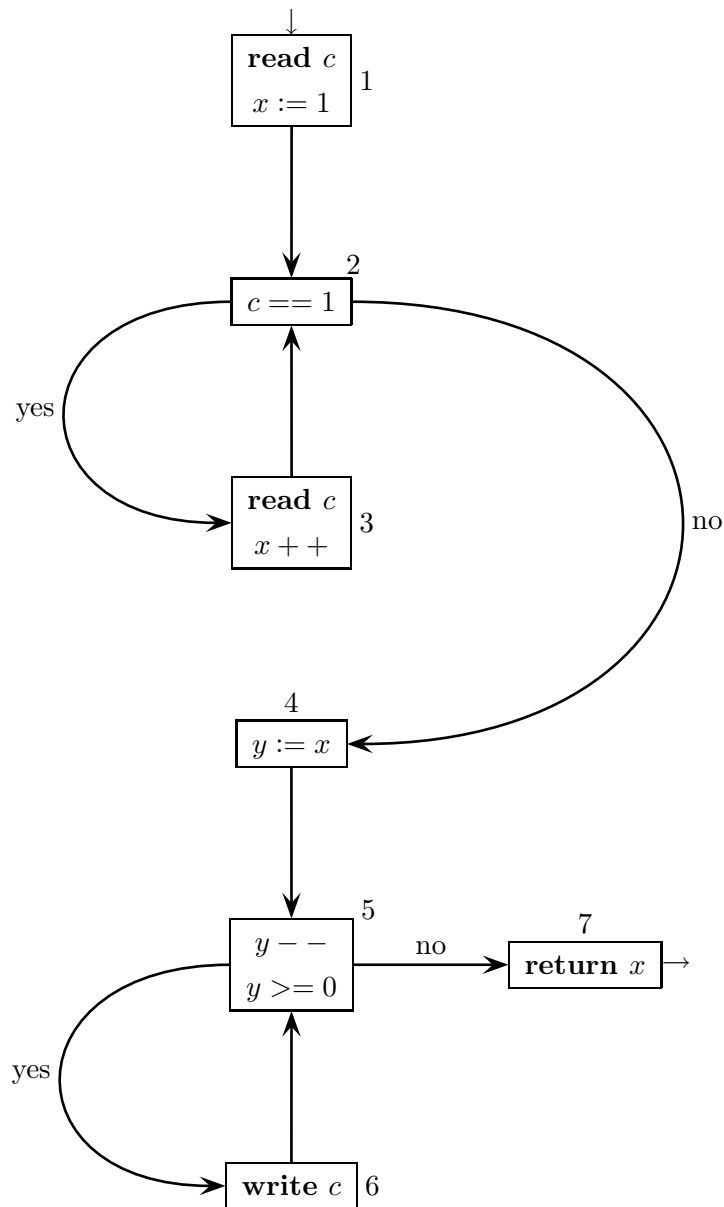
The program has three integer variables: x , y and c .

Answer the following questions (use the tables / drawings / spaces on the next pages):

- Disregarding the program semantics, write a regular expression R that represents the execution traces of the *CFG* over the node name alphabet $\{ 1, \dots, 7 \}$.
- Informally find the live variables at each node and write them by each node (the live variables of a node are those live at the node input). Can the found liveness intervals be exploited to optimize the memory allocation for the program ?
- Find the variables used and defined at each node. Then write the flow equations of the live variables, but only for the nodes 2, 5 and 6 (do not solve such equations). Verify that the solution found at point (b) satisfies the flow equations of nodes 2, 5 and 6. Remember that each node has two equations: *in* and *out*.
- (optional) Considering the *program semantics*, write an expression (not necessarily a regular expression), by refining the expression R of point (a), that defines more precisely the language of the execution traces of the *CFG*, over the alphabet $\{ 1, \dots, 7 \}$; also write a *grammar* G that generates such execution traces. If semantics is considered, do the liveness intervals of the variables change?

please write here the RE R – question (a)

please write here the live variables – question (b)



space for answering question (c)

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	

(partial) system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
2		
5		
6		

space for the equation verification

space for answering question (d)

free space for any use